

Kunming AI+

An Applied Introduction to AI with PyTorch

Luca Ghafourpour



**UNIVERSITY OF
CAMBRIDGE**
Department of Engineering

Session Title	Topics
Workshop 1	VS Code, Conda and PyTorch
Practical Workshop 1	Constructing and training a neural network from scratch
Workshop 2	PyTorch modules for building and training neural networks
Practical Workshop 2	PyTorch modules for data loading
Workshop 3	Diagnosing training performance
Practical Workshop 3	Initialisation and regularisation
Academic Lecture 1	Physics Informed Neural Networks
Academic Lecture 2	Neural Operators
Workshop 4	Convolutional neural networks
Practical Workshop 4	Scikit Learn
Workshop 5	Complete end-to-end experiment
Practical Workshop 5	Open Q&A

From scratch to abstractions

Last session we built a neural network, and designed a training algorithm by hand. Today, we instead use pre-built tools given in PyTorch.

By hand (Practical 1)

```
torch.randn(...).requires_grad_()
```

```
def forward(x): ...
```

```
((y_hat - y)**2).mean()
```

```
p -= lr * p.grad
```

```
p.grad.zero_()
```

PyTorch (Today)

```
nn.Linear
```

```
nn.Module
```

```
nn.MSELoss
```

```
optimizer.step()
```

```
optimizer.zero_grad()
```

These abstractions require less code, and result in fewer bugs.

Recap: the network from scratch

```
# Define parameters by hand
W1 = (torch.randn(1, 64) * 0.5).requires_grad_(True)
b1 = torch.zeros(64, requires_grad=True)
W2 = (torch.randn(64, 1) * 0.5).requires_grad_(True)
b2 = torch.zeros(1, requires_grad=True)

# Training loop
for epoch in range(2000):
    h = torch.relu(X @ W1 + b1)           # 1. forward
    y_hat = h @ W2 + b2
    loss = ((y_hat - y)**2).mean()        # 2. loss
    loss.backward()                       # 3. backward
    with torch.no_grad():
        for p in params: p -= lr * p.grad # 4. step
    for p in params: p.grad.zero_()       # 5. zero
```

It gives complete control, but the parameter list is ours to manage, and easy to cause errors.

nn.Module

What is an nn.Module?

nn.Module is a PyTorch subclass that combines three core things:

- **The model parameters**

Weights and biases are registered and tracked automatically. Access them all via `model.parameters()`. No need to create a manual list of parameters.

- **A forward() method**

Defines the computation on its input. Calling `model(x)` runs it.

- **Composability**

Modules can contain other modules. A whole network is just one big Module made of smaller ones.

nn.Module: the core abstraction

```
class TwoLayerNet(nn.Module):  
    def __init__(self):  
        super().__init__()  
        self.fc1 = nn.Linear(1, 64)  
        self.fc2 = nn.Linear(64, 1)  
        self.act = nn.ReLU()  
  
    def forward(self, x):  
        h = self.act(self.fc1(x))  
        return self.fc2(h)
```

nn.Linear(in dim, out dim) defines the
 $xW + b$

operation. It creates and tracks the weight and bias tensors automatically.

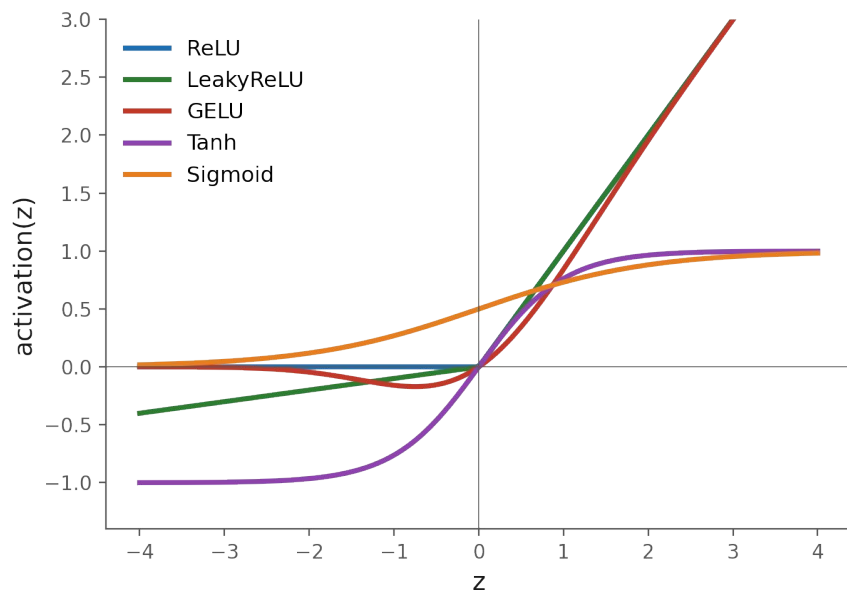
Note, no manual parameter list!
Calling **TwoLayerNet.parameters()** returns them all.

nn.Module is passed as parent class to our custom defined network module: TwoLayerNet

Activation Functions using nn

Activation functions

So far we've only considered the ReLU activation function. This is one of many!



ReLU — is cheap, but has a discontinuous kink and zero gradient for $z < 0$.

LeakyReLU — small slope for $z < 0$, so post-activated nodes never fully saturate to 0.

GELU — smooth ReLU, which has become the default in modern transformers.

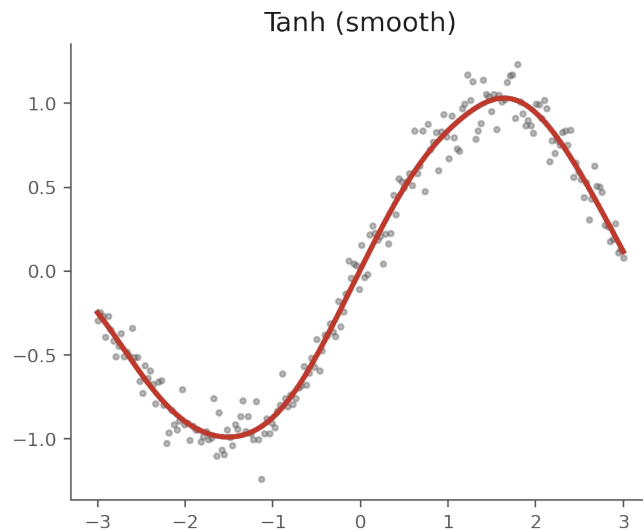
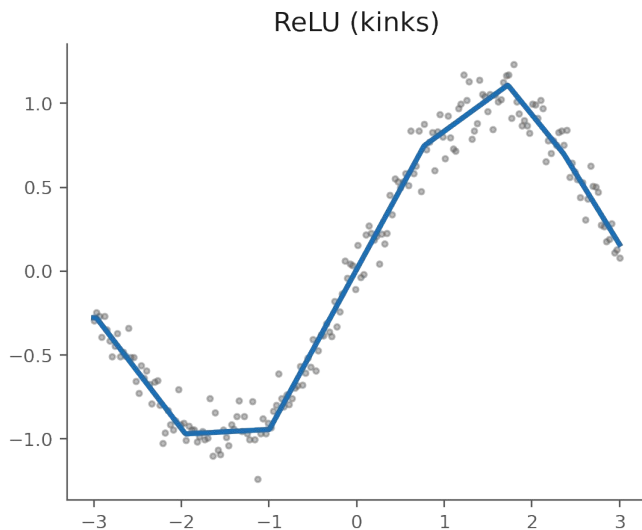
Tanh / **Sigmoid** — smooth but saturates.

Activation functions

Function	Pros	Cons	Typically used in
nn.ReLU $\text{ReLU}(z) = \max(0, z)$	cheap; gradient 1 for $z > 0$; sparse	"dead" nodes (0 grad for $z < 0$)	CNNs, default hidden layers
nn.LeakyReLU $\text{LeakyReLU}(z) = \max(\alpha z, z)$	never fully dies (slope α for $z < 0$)	extra hyperparameter α	GANs, deep CNNs
nn.GELU $\text{GELU}(z) = z \Phi(z)$	smooth; strong empirically	slightly costlier	Transformers (BERT, GPT)
nn.Tanh $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	smooth; zero-centred $(-1, 1)$	saturates; vanishing grad	RNNs, shallow nets
nn.Sigmoid $\sigma(z) = \frac{1}{1 + e^{-z}}$	output in $(0, 1)$ = probability	saturates; not zero-centred	output layer (binary class.)

$\Phi(z)$ is the Gaussian CDF. α is a small constant (e.g. 0.01).

Activation functions



ReLU is piecewise-linear. Each ReLU unit introduces one hinge, so few units give visible kinks.
Tanh is smooth, so it traces a genuinely curved fit.

ReLU's Favourable Gradients

Tanh and Sigmoid saturate: This means their gradient $\rightarrow 0$ for large inputs.

Vanishing gradient problem: Backprop multiplies gradients layer by layer, meaning near-zero values compound, and early layers stop learning.

ReLU's gradient is exactly 1 for $z > 0$. The signal passes through unchanged, regardless of depth.

Rule of thumb:

- Use ReLU or GELU for hidden layers
- Use sigmoid or softmax only at the output for probabilities.

Optimisers using torch.optim

Gradient Descent

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(\theta_t)$$

- Updates the gradient deterministically over the full dataset



Guaranteed reduction in loss at each step for small enough η



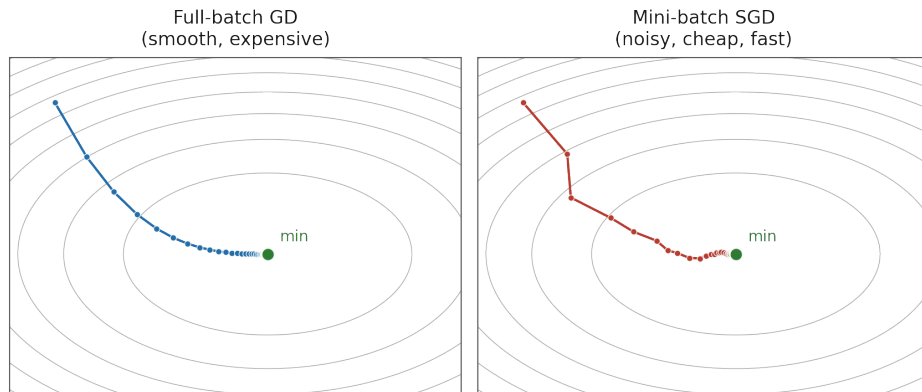
Computing the gradient over millions of examples for each step is very slow.

Stochastic Gradient Descent

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(\theta_t; \mathcal{B}_t)$$

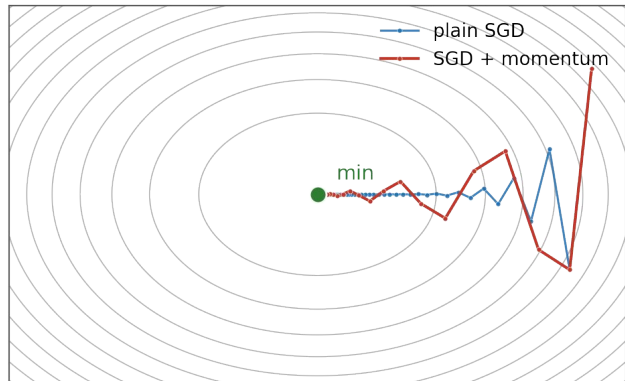
Estimates the gradient over mini-batches $\mathcal{B}$

- ✓ Mini-batches fit in memory, while full batches often do not.
- ✓ Parameters are updated after each mini-batch, giving multiple updates per epoch.
- ✓ The gradient is noisier, but this noise can help escape sharp minima, poor local minima, and saddle points.



Stochastic Gradient Descent with Momentum

$$v_{t+1} = \gamma v_t + \nabla_{\theta} L, \quad \theta_{t+1} = \theta_t - \eta v_{t+1}$$



Idea: Momentum keeps a velocity, v , which acts as a running average of past gradients.

γ controls how much previous velocity is retained. A typical value is 0.9

Effect: Consistent gradient directions accumulate, while noisy oscillations cancel out.

Momentum reduces zig-zagging in steep directions and accelerates progress along flatter ones.

Adaptive Moment Estimation (Adam)

Combines momentum with a per-parameter adaptive step size.

1. Track running averages of the gradient and squared gradient:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

2. Update parameters using the bias-corrected estimates:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$$

Adam takes smaller steps for parameters with consistently large gradients, and larger steps for parameters with smaller gradients. Typical values **$\beta_1=0.9$, $\beta_2=0.999$**

Learning rate

The learning rate (η) defines how big a step we take in parameter space in the direction of the negative gradient of the loss.

η too small training is extremely slow.

η too large, parameter updates overshoot, oscillate, or diverge to NaN.

Different optimisers want different rates. Adam usually works better with a smaller η than SGD.

Practical advice: start with Adam at $\text{lr} \approx 1\text{e-}3$, then tune.

Loss functions using nn

Loss functions

The loss function measures how wrong the model is. PyTorch provides common losses in `torch.nn`.

Function	When to use	Output layer	Target type
<code>nn.MSELoss</code> $\text{mean}((\hat{y} - y)^2)$	Regression	None (raw)	Continuous
<code>nn.L1Loss</code> $\text{mean}(\hat{y} - y)$	Regression (robust to outliers)	None (raw)	Continuous
<code>nn.CrossEntropyLoss</code> Softmax + negative log-likelihood	Multi-class classification	Raw logits	Class index

Combining

The new abstracted training loop

```
model    = TwoLayerNet()
loss_fn  = nn.MSELoss()
opt      = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)

for epoch in range(2000):
    y_hat = model(X)           # 1. forward
    loss  = loss_fn(y_hat, y)  # 2. loss
    opt.zero_grad()           # 5. zero
    loss.backward()           # 3. backward
    opt.step()                 # 4. step
```

The original fourteen line “5-step training loop” just became five lines!

Time for some code...